Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

Exam simulation

- Industries: Appliances (not better specified, they could be coffee machines, ice cream machines, refrigerators, ...)
- Business model: B2B
- Digital environments:
  - ERP (legacy monolithic system; SOAP)
    - warehouse
    - pricelists (one per country)
    - products' technical information
  - CRM (cloud, saas, restAPI)
  - E-commerce (cloud, saas, restAPI and CSV exchange)
  - PIM (on premise, restAPI and CSV exchange)

Exam simulation

- Requested goal for the business: obtain data about utilization of appliances (uptime, cicle of utilizations, kind of products used within the appliance, user identification)
- Business goals:
  - track users' data to schedule some marketing campaigns
  - track users' data to understand the taste of customers
  - track users' data to offer suggestions and new values to customers
  - track appliances information about usages to predictive maintenance
- Constraints:
  - no specific constraints

Exam simulation

- Exam tasks:
  - define the draft of architecture
  - setup the project (software repo, documentation structure, methodology, deliverable, tools, ...)
  - add details about how to implement a success solution

- Hints:
  - keep it simple
  - if something is not specified, make an assumption and write it down
  - more than one solution exists to reach the goal, choose the one you like more
    - pay attention: it has to work (at least on the paper)

Resolution

First step

- The first step is, as usual, understand the context, understand the request, sort the elements and think about a solution.

- Is everything clear?

Ca' Foscari University of Venice

First step

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
THPAPD

Ca' Foscari University of Venice

Overview the process

from previous lectures

Get requirements

business
functional
no-functional

topic covered in Software Architecture and Software Engeneering

Design the program

choose the language
choose the technologies
check the constraints
producing technical analysis
define the architecture
define software life-circle

Draw the architecture

draw the architecture
draw system landscape

considering all constraints

Define/find/ schedule/organize resources (Human and not Human)

people skills
budget
tools that we already have
tools that we have to buy
define the methodology to use based on all information above

Kickoff the project

when everything is defined, we can start with the project with the setup of the environments, developing tools and setup the methodology

Overview the process

from previous lectures

Get requirements

~~business~~
~~functional~~
~~no-functional~~

topic covered in Software Architecture and Software Engeneering

→ not necessary

Design the program

~~choose the language~~
choose the technologies
check the constraints
~~producing technical analysis~~
define the architecture
define software life-circle

Draw the architecture

draw the architecture
draw system landscape

~~considering all constraints~~

Define/find/ schedule/organize resources (Human and not Human)

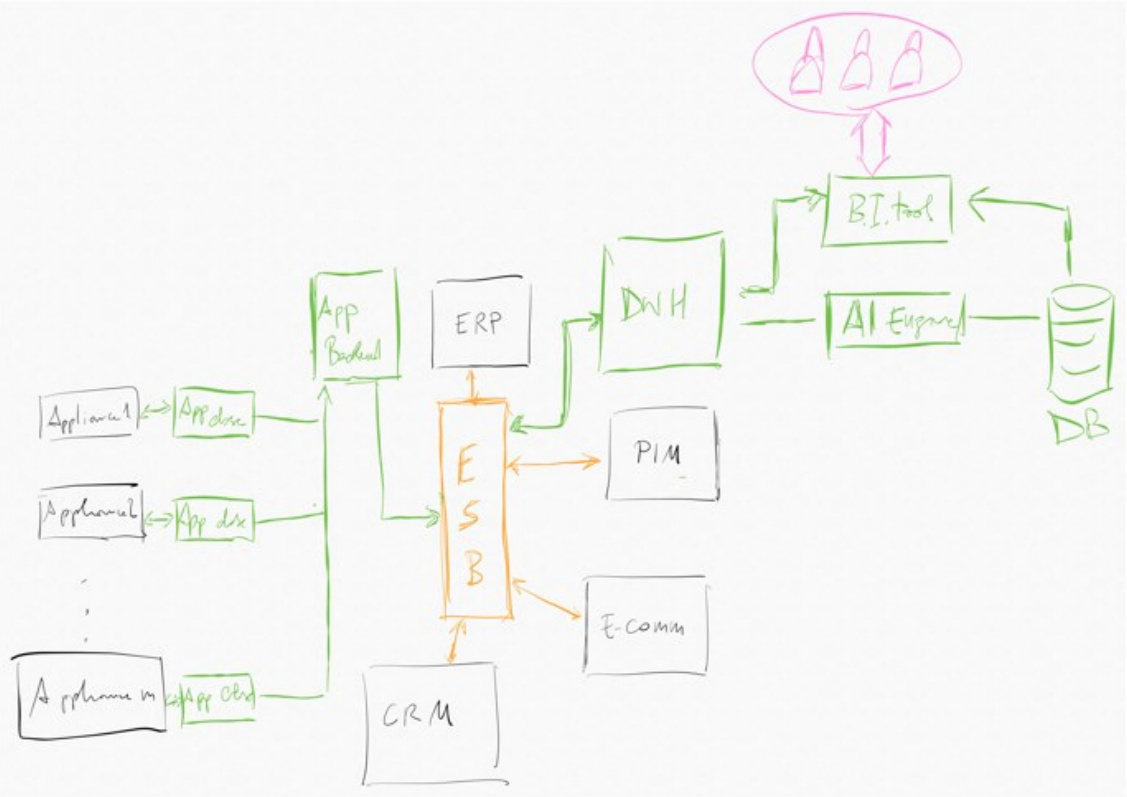
people skills → people that we need
~~budget~~
tools that we already have → tools that we need
~~tools that we have to buy~~
define the methodology to use based on all information above

Kickoff the project

when everything is defined, we can start with the project with the setup of the environments, developing tools and setup the methodology

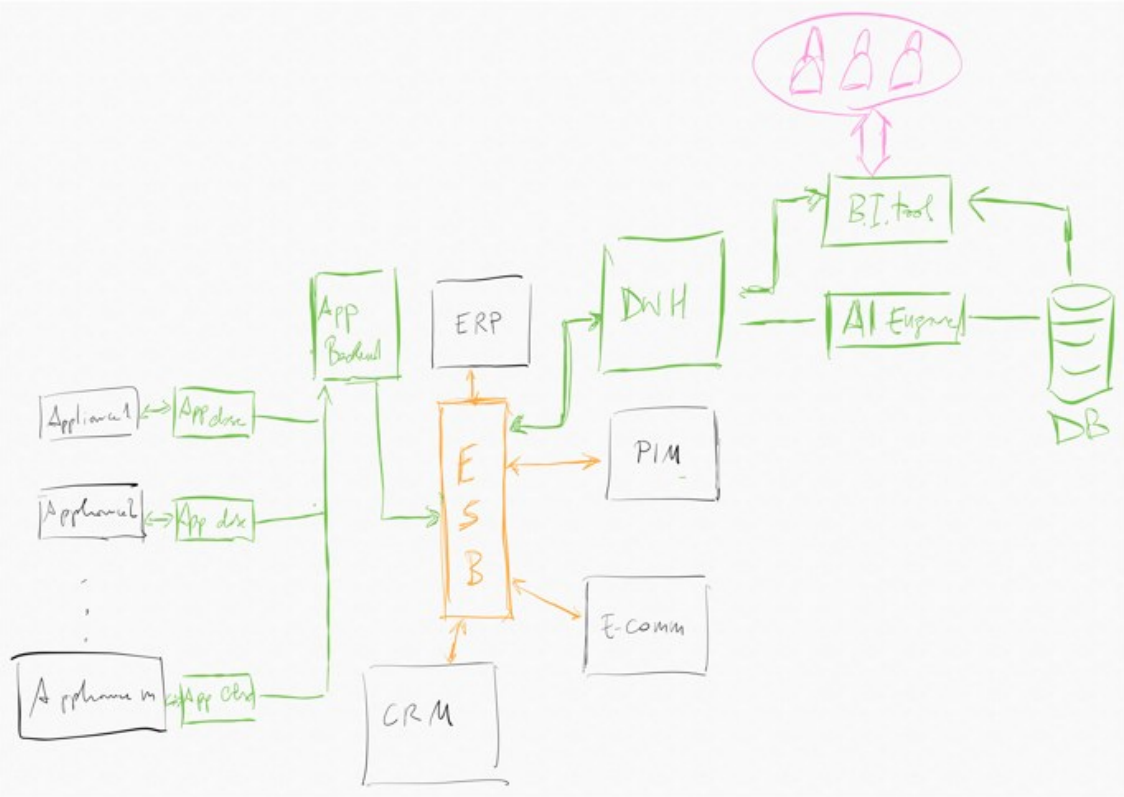
Ca' Foscari University of Venice

Possible architecture



- ESB presence is an assumption based on the presence of legacy systems and cloud solutions on the same environments.
- App backend is the software that collect all data from each appliance and move them to the specific master data.
- App close is an application that is close to the installation (could be a mobile app, a software within the appliance or a machine installed near the device) which collect the data from sensors (or in other way) and send information to the "App backend".
- DWH where all data are stored in order to analyze data for usages and forecasting.

Possible architecture



- Variances
  - No ESB, point to point connections
  - No DWH, App Backend store all the data and analysis are performed directly through App Backend (or connecting directly to the DB of the application)
  - No App close, you can assume that to use the appliance you have to use a pc or tablet where you fill all the details and send the information to the centralized server
  - Adding some improvement likea Queue manager for each "App close" in order to manage the case the customer does not have a stable connection

Ca' Foscari University of Venice

Software architecture

- Assumptions:
  - the appliances are connected to internet through WiFi connection
  - the whole team has the skills and the autonomy to work on everything
  - the appliances are always the same (we are considering just one products, no more)
    - that means that when we implement all the flow for one appliance, we do not need any rework on others
  - All services are based on AWS, then we proceed also in this case with AWS

- App close: installed on a small pc close to the machine in order to save all data.
  - it contains a server web for dashboard, control panel, spare parts commerce connection
  - it contains RabbitMQ for queue management
  - it is designed in order to manage more than one appliances at the time
  - it will work also off-line: it will save data form appliances and when the application will come back online it will send all the messages following the queue
  - app close is a microservice application, each service will be managed by a different process
  - it manage notifications (pushed by app backend)

Ca' Foscari University of Venice

Software architecture

- App backend will receive and send information from and to app close
  - receive data to track (that will be pushed on different target, depends on the type of data)
  - push updates for app close (for software within app close and for appliances, too)
  - push information about spare parts
  - push alert to app close, in case it was necessary (e.g., alert for book an appointment for maintenance)
- ESB will be updated in order to managing normalization of the data based on the target
- DWH will receive and store all the data that App backend receive from different app close.
- AI engine (and related application) in order to read from DWH to produce some prediction/forecasting for the machine
  - these information will be sent and saved to App backend and then pushed to the App client
  - these information will be forwarded to the internal R&D team, too – to act internally for product improvement
- For BI tools (Business Intelligence) we keep simple using a market tool like MS Power BI or other – not relevant the name

Ca' Foscari University of Venice

Teams to work on the project

- Software/firmware on appliance (in order to collect data through wifi) team
- App close team
- App background team
- ESB team to modify and add some flows in order to track partial data also in CRM (e.g., usage information by the customer), ERP (e.g., enhance product information related to appliance fault) and e-commerce (e.g., a purchaise for spare parts)
  - assumption: we will use some already-existing interfaces
- DWH team
  - data governance
  - data organization
  - model definition
    - interfaces definition/implementation
- BI system
  - in order to setup view and shared dashboard
- AI engine and application
  - installation and configuration (plus training) of a private LLM
  - building an app that read the data and execute some computation in order to define some forecasting through using LLM
  - DB that saving report and forecasting

Ca' Foscari University of Venice

Methodology

- Agile methodology could be a good solution – no preferences for the methodology
- 2-3 weeks sprint should be enough
- sprint review with all the teams in order to share the progress of various streams
- kanban dashboard for a complete overview of the project

Necessary tools

- Communication tools
  - not all the team members and, for sure, not all the teams will be in the same office
- Documentation tools
  - some tools to write requirements and updates
  - some tools to organize documents
- Activities backlog
  - some tools to track backlog activities or issue/bugs that will rise during the tests

Ca' Foscari University of Venice

Software lifecycle

- standard 3 environments
- 2 main branches: main and development
  - main -> branch that collect all the implementation and will be used to create a release (which will be deployed in staging environment)
  - development -> branch that will be automatically deployed in development environment
- CI/CD that build and deploy in development environment at each merge in development branch
- terraform for the cloud services configuration
- ansible in order to perform fast installation
  - useful also for production (e.g., application close that will be instantiated to each site of customer)
- testing
  - applicance with different firmware version in order to test compatibility of new updates of software
  - unit tests for main functions
  - integration tests for all flows and for majority of cases (happy path, wrong path, error handling, ...)
  - end2en cases only for app close and AI engine and applications

Internal note – what we can skip

- CDN
  - no content distribution needs
  - we can evaluate it for security reason, but it is not mandatory
- Redis
  - no specific needs of Redis (and in memory DB)
  - we can use different products since we do not need fast access to the data
- Data Mesh
  - we have a DWH where we can store every information

Adding valuable topic

- Definition of Data Model to exchange data
- Definition type of communication among systems
  - e.g., restAPI with asynchronous connection to move data to the DWH (in order to have fresh data)